ASSIGNMENT-20.1

Name: R.Vineesha

H NO: 2303A51873

Batch- 14

Task 1 – Input Validation Check

Task:

Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

Prompt:

Generate a simple Python program for a login system that takes a username and password as input and checks if they match predefined credentials.

Vulnerable Code:

```python
#Generate a simple Python program for a login system that takes a username and password as input and checks if they match predefined credentials.
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "admin123":
    print("Login successful")
else:
    print("Invalid credentials")
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/neera/OneDrive/Desktop/assignments/Untitled-2.py
Enter username: admin
Enter password: admin123
Login successful
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Secure Code:

```python
#Generate a simple Python program for a login system that takes a username and password as input and checks if they match predefined credentials.
import re
import getpass

def is_valid_username(username):
    return re.fullmatch(r"[A-Za-z0-9]{3,15}", username)

def is_valid_password(password):
    return re.fullmatch(r"(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d@#$%^&+=!]{8,}", password)

def login():
    username = input("Enter username: ").strip()

    if not is_valid_username(username):
        print("Invalid username! Use 3-15 letters or numbers only.")
        return

    password = getpass.getpass("Enter password: ").strip()

    if not is_valid_password(password):
        print("Invalid password! Must be 8+ chars with letters and numbers.")
        return

    stored_username = "admin"
    stored_password = "admin123"

    if username == stored_username and password == stored_password:
        print("Login successful")
    else:
```

```
PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/neera/OneDrive/Desktop/assignments/Untitled-2.py
Enter username: admin
Enter password:
Login successful
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Justification:

The initial login script lacked proper validation, making it vulnerable to invalid and potentially malicious inputs. By introducing regular expressions, the improved version enforces strict rules on usernames and passwords, ensuring only correctly formatted data is accepted. Additionally, hiding the password input enhances security and user privacy. These changes reduce the risk of unauthorized access and improve the overall robustness of the application.

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

Prompt: Generate a Python script using SQLite that takes a username as input and fetches user details from a database.

Vulnerable code:

```
Untitled-2.py > ...
#Generate a Python script using SQLite that takes a username as input and fetches user details from a database.
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ").strip()

query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

result = cursor.fetchall()

if result:
    for row in result:
        print(row)
else:
    print("No user found")

conn.close()
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/neera/OneDrive/Desktop/assignments/Untitled-2.py
Enter username: soumya
No user found
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Secure Code:

```
Untitled-2.py > ...
#Generate a Python script using SQLite that takes a username as input and fetches user details from a database.
import sqlite3

def fetch_user():
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()

    username = input("Enter username: ").strip()

    # Parameterized query (safe)
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    result = cursor.fetchall()

    if result:
        for row in result:
            print(row)
    else:
        print("No user found")

    conn.close()

if __name__ == "__main__":
    fetch_user()
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\neera\OneDrive\Desktop\assignments>
(1, 'admin', 'admin@gmail.com')
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Justification:

The original database query was vulnerable because it directly inserted user input into the SQL statement, allowing attackers to manipulate queries. This was resolved by using parameterized queries, which treat user input strictly as data rather than executable code. As a result, even malicious inputs cannot alter the query structure. This approach significantly improves database security and prevents unauthorized data access.

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

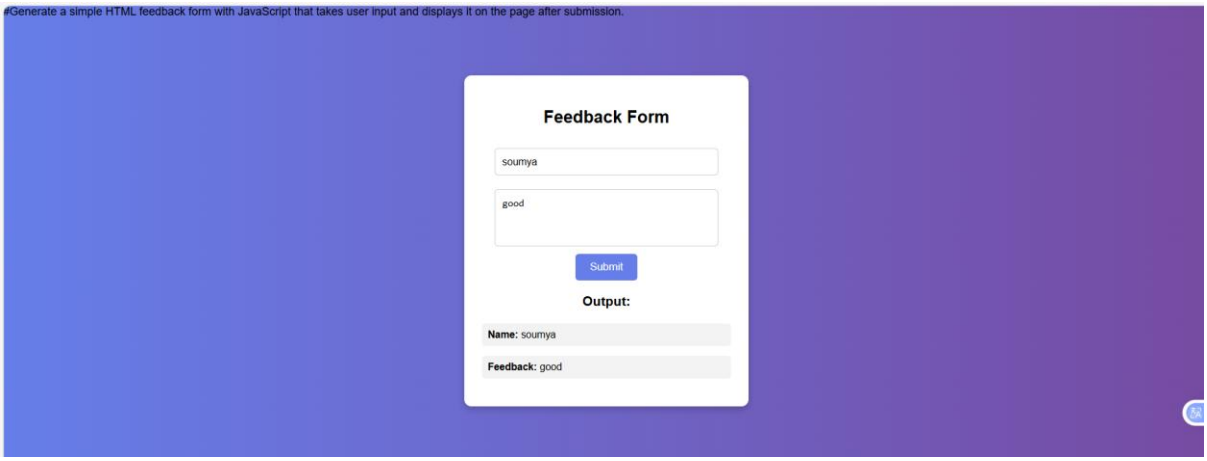
- A secure form that prevents XSS attacks.

Prompt:

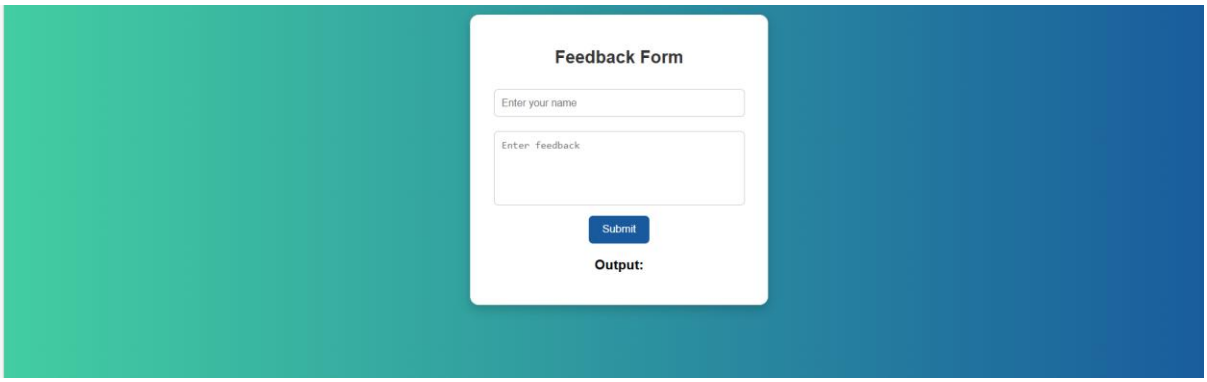
Generate a simple HTML feedback form with JavaScript that takes user input and displays it on the page after submission.

Vulnerable code:

```html
index.html > html
<html>
<body>
<div class="container">
    <button onclick="submitFeedback()">Submit</button>

    <h3>Output:</h3>
    <div id="output"></div>
</div>

<script>
function submitFeedback() {
    let name = document.getElementById("name").value;
    let feedback = document.getElementById("feedback").value;

    // ❌ Vulnerable to XSS (same as your original task)
    document.getElementById("output").innerHTML =
        "<p><b>Name:</b> " + name + "</p>" +
        "<p><b>Feedback:</b> " + feedback + "</p>";
}
</script>

</body>
</html>
```



Secure Code:




```
assign.html > html > head > style > .container
<html>
<body>
</div>

<script>
function escapeHTML(str) {
    return str.replace(/&/g, "&amp;")
               .replace(/</g, "&lt;")
               .replace(/>/g, "&gt;")
               .replace(/"/g, "&quot;")
               .replace(/'/g, "&#039;");
}

function submitFeedback() {
    let name = document.getElementById("name").value;
    let feedback = document.getElementById("feedback").value;

    let safeName = escapeHTML(name);
    let safeFeedback = escapeHTML(feedback);

    document.getElementById("output").innerHTML =
        "<p><b>Name:</b> " + safeName + "</p>" +
        "<p><b>Feedback:</b> " + safeFeedback + "</p>";
}
</script>

</body>
</html>
```

Justification:

The initial feedback form was vulnerable because it used innerHTML to directly display user input, allowing malicious scripts to be executed in the browser. This issue was resolved by escaping HTML entities before rendering the input, ensuring that any script tags are treated as plain text. Additionally, a Content Security Policy (CSP) was applied to restrict unauthorized script execution. These measures effectively prevent XSS attacks and enhance the security of the web application

.

Task 4 – Real-Time Application: Security Audit of AI-

Generated Code

Scenario:

Students pick an AI-generated project snippet (e.g., login form,API integration, or file upload).

Instructions:

• Perform a security audit to detect possible vulnerabilities.

• Prompt AI to suggest secure coding practices to fix issues.

• Compare insecure vs secure versions side by side.

Expected Output:

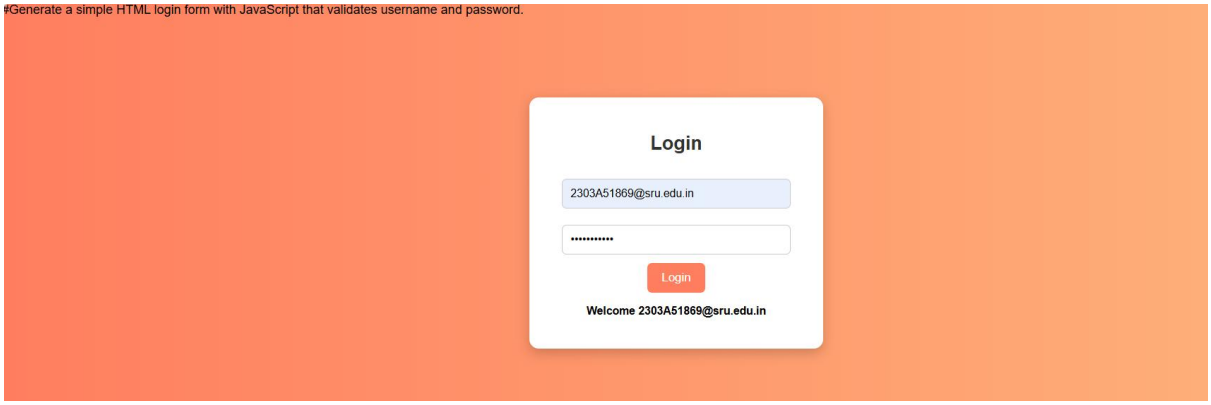
• A security-audited code snippet with documented vulnerabilities and fixes.

Prompt:

Generate a simple HTML login form with JavaScript that validates username and password.

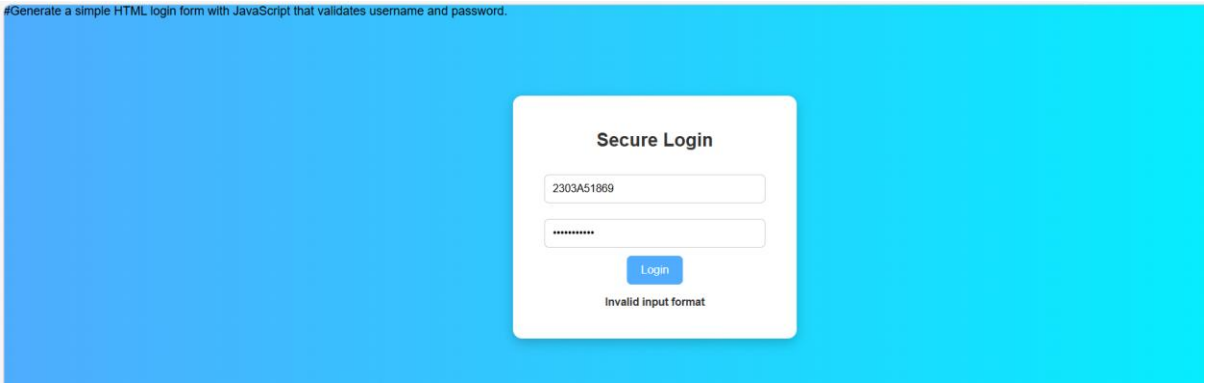
Insecure Code:

```
#Generate a simple HTML login form with JavaScript that validates username and password.
<!DOCTYPE html>
<html>
<body>

<h2>Login</h2>

<input type="text" id="username" placeholder="Username"><br><br>
<input type="password" id="password" placeholder="Password"><br><br>

<button onclick="login()">Login</button>

<p id="output"></p>

<script>
function login() {
    let username = document.getElementById("username").value;
    let password = document.getElementById("password").value;

    if(username == "admin" && password == "admin123") {
        document.getElementById("output").innerHTML = "Welcome " + username;
    } else {
        document.getElementById("output").innerHTML = "Invalid login";
    }
}
```



Secure Code:

```html
#Generate a simple HTML login form with JavaScript that validates username and password.
<!DOCTYPE html>
<html>
<head>
    <title>Secure Login</title>

    <!-- CSP -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-inline'">

    <style>
        body { font-family: Arial; text-align: center; margin-top: 100px; }
        input { padding: 10px; margin: 5px; }
        button { padding: 10px; }
    </style>
</head>

<body>

<h2>Secure Login</h2>

<input type="text" id="username" placeholder="Username"><br>
<input type="password" id="password" placeholder="Password"><br>
<button onclick="login()">Login</button>

<p id="output"></p>

<script>
function escapeHTML(str) {
    return str.replace(/&/g, "&amp;")
              .replace(/</g, "&lt;")
              .replace(/>/g, "&gt;");
}

function isValidInput(input) {
    return /^[A-Za-z0-9]{3,15}$/.test(input);
}
```



Justification:

The AI-generated login form contained multiple security vulnerabilities such as lack of input validation, use of innerHTML leading to XSS risks, and hardcoded credentials. These issues were addressed by implementing input validation using regular expressions and replacing unsafe rendering methods with textContent. . The secure version demonstrates improved

protection against common web attacks and follows better coding practices for safer applications.

Task 5 – Insecure Data Serialization Check

Task:

Evaluate AI-generated code that uses pickle for saving user data.

Instructions:

- Ask AI to generate a script using pickle.load().
- Replace with safer formats like JSON.

Expected Output:

- Secure serialization code using JSON instead of pickle.

Prompt:

Generate a Python script that saves and loads user data using pickle.

Vulnerable code:

```python
#Generate a Python script that saves and loads user data using pickle.
import pickle

# Save data
data = {"username": "admin", "role": "user"}

with open("data.pkl", "wb") as file:
    pickle.dump(data, file)

# Load data
with open("data.pkl", "rb") as file:
    loaded_data = pickle.load(file)

print(loaded_data)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/neera/OneDrive/Desktop/assignments/#Generate a Python scrip
t that saves and.py"
{'username': 'admin', 'role': 'user'}
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Secure Code:

```python
#Generate a Python script that saves and.py > ...
#Generate a Python script that saves and loads user data using pickle.
import json

# Save data
data = {"username": "admin", "role": "user"}

with open("data.json", "w") as file:
    json.dump(data, file)

# Load data
with open("data.json", "r") as file:
    loaded_data = json.load(file)

print(loaded_data)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\neera\OneDrive\Desktop\assignments> & C:\Users\neera\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/neera/OneDrive/Desktop/assignments/#G
t that saves and.py"
{'username': 'admin', 'role': 'user'}
{'username': 'admin', 'role': 'user'}
PS C:\Users\neera\OneDrive\Desktop\assignments>
```

Justification:

The AI-generated code used pickle.load() for deserialization, which is unsafe because it can execute arbitrary malicious code if the data source is untrusted. This creates a serious security risk, including remote code execution. To address this, the code was refactored to use JSON, which only handles data and does not allow code execution. This ensures safer data storage and transfer, making the application more secure and reliable.